The SWIRL Assets Compiler

The compiler translates **SWIRL**'s simple *assets-description language* file (`.sadl`) into **Rust** files. The generated code takes care of:

- Generating *hierarchical* `enum`erations and `struct`ures. The structures can be queried through members or indexed through the various `enum`s.
- Generating code to load all assets, returning the top-level `struct`.
- Generating an initial scene graph

For the time being, the compiler is only aware of *font*, *audio* and *image* formats.

- Fonts should be `.woff` files, as this format is universally recognized by all major browsers.
- When provided a single audio file with the extension `.mp3` or `.ogg`, it will automatically include a file with the other extension as an alternative.
- Images are currently limited to `.jpg` and `.png` types. It is possible to specify a numerically-tagged *range* of images, which the compiler will use to create a `SpriteMovie`.

This project began as a Rust *procedural macro*; however, letting users see the generated code was deemed necessary for optimal usability and easier learning. Users can inspect it and *see* types and declarations instead of having to remember and apply the compiler's name-generations rules every time.

Building

See BUILDING for compiling instructions.

Syntax

The syntax of `.sadl` files is rather simple. Spaces, tabs and newlines are ignored; the language is *free-form*, letting users choose whichever formatting convention they prefer. It is, however, **case-sensitive**.

Line comments in Rust-style (`//`) and Haskell-style (`--`) are both allowed. They're ignored, and absent from the compiler's output.

Every file is divided into two main sections: `assets` and `graph`. To begin a section, just write the section name followed by a colon:

```
assets:
```

The `assets` section is mandatory; `graph` is optional.

`assets` Section

The assets section is a hierarchical description of files to be loaded:

```
assets:

    OuterGroup {
        SimpleElement = "filepath.jpg",
        RangedElement = "prefix" + [1 ..= 25] + "postfix.jpg",
        GroupElement {
            InnerSimpleElement1 = "another-filepath1.jpg",
            InnerRangedElement = "another-prefix-" + [ .. 1000 : 3] + ".png"
            InnerSimpleElement2 = "another-filepath2.woff",
            InnerGroupElement {
                InnermostSimpleElement1 = "whatever.mp3",
            },
        },
    }
```

Roughly, this section provides the ability to describe a hierarchical set of *named* resources and categories. There are *groups* and every group has several *elements* specifying one or more files, or functioning as groups for a deeper organizational hierarchy.

In the target files, groups are realized as `struct`s and `enum`s, and elements become *fields* and enumeration *variants*. The `struct` hiearchy will be used to load and contain the actual resources, and the `enum`s will be used as indexers to the former.

There are few rules regarding valid names:

- All names must be in **PascalCase** (aka **CamelCase**.) Only letters and digits may be used, and every name must begin with an uppercase letter. Generally, every consecutive word is supposed to start with an uppercase letter as well, but the compiler won't check for this (that's too anal-retentive even for the author.)
- There must be a single topmost group (in this example, `OuterGroupName`.)
- Names must **not** end in `Struct` or `Set`. These are appended to group names for enumeration and record names.
- Name duplication is forbidden, even at different levels.

Groups are made up of elements. *Simple* elements refer to a single file (unless it's a sound file, in which case it references two.) *Range* elements refer to a list of files using identical prefixes and postfixes, but with a different numbers between those. Finally, *Complex* elements specify new groups, allowing for more hierarchy levels.

Simple elements The simplest item definitions are those followed by an equal sign and a **valid** filename (it must have a loadable extension, such as `.jpg` or `.woff`.)

```
SimpleElement = "filepath.jpg"
```

If the extension of the filename happens to be `.ogg` or `.mp3`, it will be identified as a sound and generate code to load a sound using either extension, allowing the browser to pick the one it supports the best.

The name of the variant would be the same, in this case as part of the `OuterGroupSet` enumeration. The *field* name would be the *snake-cased* version, i.e., `simple_element` (all lowercase, with underscores between word limits), which in this case would be part of a `OuterGroupStruct`.

File names (and parts of names) are double-quoted, and there are restrictions on valid characters – namely, any *printable* character can be used except for the following:

```
< > # % @ $ & + * ? ! , ; : / \ | [ ] { } ' " `
```

The rule attempts to prevent the resulting URLs from transcription errors or file system rejection.

Ranged elements These specify a range and optional zero-padding. These are reserved for image sequences, and will be converted into `SpriteMovie`s. Both **Rust** range operators (`..`, `..=`) can be used to specify a range. Example:

```
RangedElement = "prefix" + [1 ..= 25] + "postfix.jpg"
```

This will load a sequence of files named `prefix1postfix.jpg` through `prefix25postfix.jpg`. In this case, the last number is included in the range by the `..=` operator.

For zero-padded numbering, simply add a colon and the total number of digits:

```
InnerRangedElement = "another-prefix" [ .. 1000 : 3] ".png"
```

As shown above, the initial number can be omitted and assumed to be zero. The resulting files would go from `another-prefix-000.png` to `another-prefix-999.png`. The `..` operator excludes the upper limit.

Regarding generated code, `RangedElement` would keep its name as a variant, and become field `ranged_element` for its immediate parent, `OuterGroupStruct`. It would also generate its own enumeration and record types, `RangedElementSet` and `RangedElementStruct`. The set would contain variants `RangedElement1` through `RangedElement25` (zero-padding or absence thereof would be respected here as well), and the record would have fields `ranged_element1` to `ranged_element25`.

Complex elements (aka Group Elements) These elements are groups that contain further elements. The leading identifier is followed by an opening brace (`{`), a list of definitions (simple, ranged or complex) separated by commas, and the closing brace (`}` .)

```
GroupElement {
    InnerSimpleElement1 = "another-filepath1.jpg",
    InnerRangedElement = "another-prefix-" + [ .. 1000 : 3] + ".png"
    InnerSimpleElement2 = "another-filepath2.woff",
    InnerGroupElement {
        InnermostSimpleElement1 = "whatever.mp3",
    },
},
```

In a very Rust-like way, the last definition inside a group may be followed by a trailing comma. There's no pre-established limit to the number of groups that can be nested inside one another.

`graph` Section

The graph section has not been implemented yet. Currently, the keyword is ignored; nothing but whitespace or comments can come after it, or (in its absence) after the `assets` section.